

url4 topology — endpoint, node, discovery, addressing, transport

Sharp definitions before the Engine grows further

STATUS	proposed — owner review; one decision left open in §5; reconciled 2026-09-15 with Kevin's Part A §1.4 refresh, PR #19 commit 2a939bff (draft, unmerged) — see §12 and Appendix D
WORK ITEM	OME-1110
DATE	2026-09-04
OWNER	Sergey Bershadsky (execution and telemetry architecture)
GRAMMAR OWNER	Kevin McDonough (URL4 spec, Parts A/B)
SPEC POINTERS	URL4.ai Specification v0.5 DRAFT — Part A §1.4 at PR #19 (2a939bff), Parts A/B on main, Parts C–I drafts on url4-refactor

Read §0 first. Everything else is the reasoning behind one line of that table.
Open questions and spec deltas live in the companion, open-work PDF.

0. The eight answers

This document fixes the words we use for the pieces of url4. It is short on purpose. Each section states a definition, says where the spec agrees or is silent, and stops.

Reconciled 2026-09-15. Kevin's Part A §1.4 refresh — `screamingface-design` PR #19, commit `2a939bff`, still a **draft** — adopted most of what this document proposed, so much of what follows now cites the spec rather than arguing with it. Where the two differ, Kevin's text wins and the difference is recorded as a delta. §12 lists what changed; Appendix D is the term-by-term crosswalk.

#	QUESTION	ANSWER
1	What is an endpoint? What is a node?	An endpoint is an interface on a node that evaluates url4, named by an endpoint path (<code>/claude</code>): it takes <code>(context)!intent</code> , returns a result, and can evaluate a full expression. It is stateless (§3). A node is the origin that serves a set of endpoints, <code>/</code> being its default.
2	Who owns what?	The node owns discovery, credentials and outbound traffic (§4, §11). Endpoints own nothing but their function. Every requestor that runs an evaluator is itself a node (§4, §13).
3	Swarm? Composition?	Not protocol words. The expression is the composition. The nodes it touches are its request tree. "Swarm" is only a deployment word for one operator's nodes.
4	<code>.well-known</code> or OPTIONS?	Three mechanisms now: the node document (Part G §27.1), the Capabilities response header (also §27.1), and OPTIONS (ours). §5 documents all three; the owner picks there.
5	<code>/name</code> vs <code>url4://name</code> ?	<code>/name</code> is an endpoint on the node that is evaluating. <code>url4://name</code> is node <code>name</code> 's default endpoint <code>/</code> . Spec Part B §5.4 already says this.
6	Where does an ensemble run?	Wherever an evaluator runs: the SDK on a laptop, or a node. A local node may mount remote endpoints under local names (proxy mounts).
7	Can I test a node first?	One fetch of the node's capabilities document, or one OPTIONS per endpoint (§5). A dry run — evaluate without executing, envelope and no result — is now the spec's own word (Part A §1.4.3); only its shape is open (§10).
8	Streaming fallback?	One GET asks for the richest mode; the endpoint answers with the best it has: WebSocket, then SSE, then sync (§6). Sync is the only MUST. The spec now carries this as the response ladder and lists <code>websocket</code> as a fourth delivery value (Part A §1.4.4); Part C has yet to catch up.
9	Text in, text out?	A habit, not a rule. Every edge carries a typed payload (text, image, audio, video, embeddings) named by its media type. Text is the default. No grammar change (§8).

What changes against today

- The Engine becomes a url4 **node**. Today no url4 endpoint is reachable over HTTP; the SDK's endpoint-to-endpoint code is unused.
- Every endpoint speaks the same GET. The endpoint picks the richest delivery it supports: **WebSocket → SSE → sync**, in one round trip. Sync is the only MUST.
- We keep the spec's two words: a **node** is an origin that serves a set of **endpoints** (`/claude`, `/codex`, ...). No rename is asked of Kevin. Part A §1.4.2 has since added a **host system** — the machine a node runs on — which we do not need and do not use; §1 says why "host" stays out of this document.
- An endpoint is a **universal inference processor**: image generation, speech, video and multimodal ensembles become ordinary expressions on the same engine, telemetry and cost accounting (§8).

- **Most of this is no longer a proposal.** `Mount`, `Evaluator`, `Dry run`, `Degradation`, the response ladder, `Scheme adapter`, `Flow constraints`, `Attribution`, `Run handle` and the `application/url4-envelope+json` envelope switch are all in Part A §1.4 as of PR #19. What is left open is in the companion PDF.

1. Terms

Eleven words. Each has one meaning. **Almost all of them are now the spec's own:** Kevin's Part A §1.4 refresh (PR #19, `2a939bff`, still a draft) adopted most of what this document proposed, so the "spec word today" column has largely stopped saying not defined. Appendix D records the crosswalk term by term; what follows is the short list a reader needs to get through §2–§9.

OUR WORD	MEANING	IN THE SPEC (PART A §1.4 @ 2A939BFF)	IN THE SDK	IN THE ENGINE
Endpoint	A logical interface on a node that evaluates url4, reached at an endpoint path.	§1.4.2, same word — now an interface, distinct from the path that names it	an entry in <code>Url4Node</code> 's endpoint registry	a route such as <code>/anthropic/<model></code>
Endpoint path	The URI path (<code>/claude</code>) identifying an endpoint relative to the node address.	§1.4.2, new — the spec now separates interface from path	the registry key	the route
Node	An origin that serves a set of endpoints, <code>/</code> being the default, and answers discovery.	§1.4.2, same word	<code>Url4Node</code>	the App, after this change
Node address	The node's network address: a URI scheme plus an RFC 3986 authority.	§1.4.2, new	<code>Url4Node</code> 's origin	the App's origin
Target	The absolute or relative endpoint URI an expression is addressed to: node address + endpoint path.	§1.4.3, sharpened — it was "an absolute/relative URI defining the Node + Endpoint"	<code>FetchRequest</code> 's destination	the route the Runner calls
Mount	The binding of an endpoint path to its evaluator: <code>local</code> (in-process), <code>command</code> (subprocess), <code>proxy</code> (declared target).	§1.4.3, adopted verbatim from this document (was not defined)	<code>endpoint()</code> , <code>[commands]</code> , —	in-process handlers only
Evaluator	Whatever runs an expression: resolves sources, fans out, reduces, runs the intent.	§1.4.3, adopted — and widened: it may or may not also be the intent processor	<code>url4.dag.run</code>	<code>Url4Executor</code> in the Runner
Intent processor	What turns resolved context plus intent into a result: a model call, a script, a command.	§1.4.3, same word	endpoint handler	one <code>httpx</code> call to aigateway
Requestor	Whoever submits an expression to a target for evaluation.	§1.4.3, same word	<code>Client</code>	product client

OUR WORD	MEANING	IN THE SPEC (PART A §1.4 @ 2A939BFF)	IN THE SDK	IN THE ENGINE
Request tree	The nodes and endpoints one expression touches.	§1.4.3, adopted with a change — the spec now says an evaluator may expand the tree and drops "strict tree, never a graph" (Part H §29.1). Appendix A delta 14 asks whether that is deliberate.	the compiled DAG, per endpoint	one run
Capabilities document	JSON a node publishes to say which endpoints, mounts, features, delivery modes and schemes it offers.	§1.4.5 and Part G §27.1–§27.2 — now also naming the Capabilities response header	none	none

Words we do not use in the protocol: ensembler, orchestrator, swarm, composition, fusion. "Fusion" stays product copy. Plan has left this list: the spec now defines **dry run** (§1.4.3), and §10 asks only what shape it takes.

"Host" is not one of our words. Part A §1.4.2 now defines a **host system** — a physical or virtual machine supporting one or more nodes — which is a useful deployment word and changes nothing here. But §1.4.4 and the §1.4.3 **Scheme adapter** row then use "host" to mean the node: it is the node that negotiates delivery, picks a response-ladder rung and holds scheme credentials. A host system negotiates nothing. This document keeps "host" out entirely; Appendix A delta 13 proposes the same for those four rows.

Naming note: Parts C–I on the **url4-refactor** branch still use the older **abc://** scheme and **ABC-*** headers; Parts A/B use **url4://**. This document writes **url4** and **URL4-*** throughout and assumes the rename Kevin lists as his open question 19 (Part I §43).

2. Addressing

Three address forms, plus any other scheme as data. The spec fixes their meaning in Part B §3.1.1, §3.5 and §5.4; we add one rule about the root.

Part A §1.4.2–§1.4.3 now names the parts: a **node address** is the scheme plus RFC 3986 authority, an **endpoint path** identifies an endpoint relative to it, and a **target** is the two together — the endpoint URI an expression is addressed to. The forms below are targets.

url4 topology — addressing: what you write, where it resolves

WHAT YOU WRITE

`/claude(ctx)!'go'`

same node, endpoint /claude

WHERE IT RESOLVES

Node (current, evaluating)

`/claude``/``url4://beta.example/claude(ctx)!'go'`

beta evaluates the sub-expression itself

Node url4://beta.example

`/claude``/``url4://beta.example`

= url4://beta.example/ → default endpoint

`https://data.example/rows`

not url4; fetched as data, nothing else

`https://data.example/rows`

plain HTTP read · no url4 semantics

`pg://warehouse/analytics.monthly_sales`

other scheme → node adapter

scheme adapters on the evaluating node

s3 · pg · sqlite · ... own credentials, typed result

Rule (Part B §5.4, §3.5): /path resolves on the evaluating node; url4:// names another node; every other scheme is data read by an adapter.

- `/claude(ctx)!'go'` — an endpoint on the node that is evaluating. Resolves to `url4://<current node>/claude` (Part B §5.4).
- `url4://beta.example/claude(ctx)!'go'` — an endpoint on another node. That node evaluates the sub-expression itself (Part B §3.1.1; SDK invariant OME-535).
- `url4://beta.example` — the node's default endpoint. **Our rule:** `url4://node` $\equiv$ `url4://node/`.
- `https://data.example/rows` — plain HTTP data. No url4 headers, no envelope, `@` is an error (Part B §3.5).

Root and version. `/` is the node's default endpoint. The spec makes `/v1` the protocol-version path (Part D §19.1) but its own examples already write `abc://thepost?q=...` with no path at all (Part I §41.27). We keep both: `/` serves the current version; `/v1` is an explicit alias. Two tensions to settle with Kevin: Part D §19.4 gives the `v` parameter precedence over the path, and §19.3 says a nested expression takes its version from the path it targets, which a bare node address does not carry. The SDK's `url4 serve (/v1)` and `Client` (default `/v1`) move to `/` (Appendix B).

Scheme inheritance. A bare relative data URI inherits the scheme of the request that carried it (Part B §5.4.1). Under `url4://` it is a url4 read; under `https://` it is a plain read.

Any scheme is a source — now the spec's own rule. This document proposed generalising the spec's `s3://` line into a **scheme adapter**; Part A §1.4.3 adopted it, as "a non-url4 source reader mounted on a node for fetching non-HTTP scheme data (`s3://`, `pg://`, `sqlite://`), typing its result, and advertised in the node's capabilities document", generalising Part B §3.5. The rules below therefore describe the spec, not a proposal. Two notes on the adopted row: it says the adapter uses "the **host's** own credentials" where it means the node's (Appendix A delta 13), and it carries a stray trailing period. The SDK already has the slot: `FetchRequest.kind` is `url4`, `http`, `relative`, or `other`.

```
(sales=pg://warehouse/analytics.monthly_sales,
 logo=s3://brand-assets/logo.png;accept=image/png,
 notes=sqlite:///srv/app/notes.db/notes,
 policy=https://data.example/policy.md)'Draft the Q3 update. Use the logo.'
```

Rules:

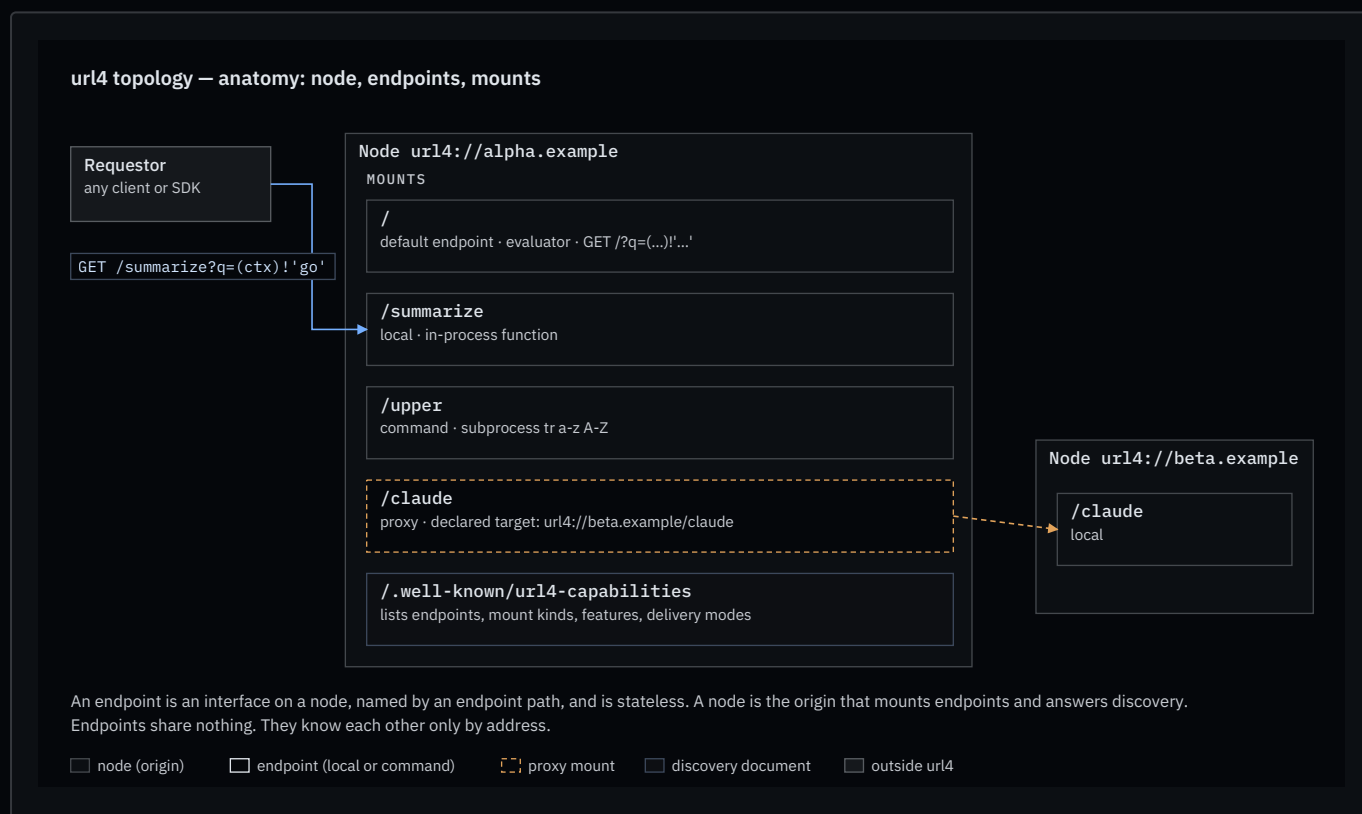
- **The evaluating node resolves it, with its own credentials.** `pg://warehouse` names a connection the node knows; the expression never carries a secret, because the expression is the shareable audit artifact (Part A §1.2). The spec's delegated-credential token can only target `abc_node` or `http_source` (Part H §31.4), so node-owned credentials are the only way a non-HTTP scheme gets any.
- **The adapter types the result (§8):** an S3 object carries its stored `Content-Type`; a table or query returns `application/json` rows, or `text/csv` when asked with `;accept`. Rows are a collection, so `pg://warehouse/analytics.monthly_sales*(row)!'summarise $item'` iterates them (Part B §5.3).
- **Advertised, or refused.** The node lists its schemes in the capabilities document; an unlisted scheme is `unsupported_mode`, permanent (Part B §3.5). Access control and consent apply as for any source (Part B §5.6.4). Such a source is not url4-aware: the envelope marks it `abc_aware: false` and attribution stops at it (Part G §26.4), the same as for any plain HTTP read.
- **Hexagonal.** A scheme adapter is an `IOLayer` adapter registered by scheme; the core never imports it. `url4 serve` gains a `[schemes]` table beside `[data]`.

What a scheme's path means (bucket and key; database, schema and table; file and table) is the adapter's contract, not the grammar's. The grammar only sees `scheme://authority/path`.

3. Endpoint contract

An endpoint is a function. That is the whole idea.

Part A §1.4.2 now says it more precisely than we did: an endpoint is a logical application interface, supporting url4 evaluation, exposed by a node and identified relative to the node address by an **endpoint path**. The interface and the path that names it are two words now, and this document follows the spec: "endpoint" for the function, "endpoint path" for `/claude`.



An endpoint:

- **MUST** answer `GET <path>?q=<expression>` and evaluate the expression. Every endpoint evaluates url4; an endpoint may walk a DAG before it reaches its own intent processor. There is one grade of endpoint, not two.
- **MUST** be stateless per invocation. It keeps nothing between calls. Its own data is reached through `@` (Part B §5.6) and lives behind it, not in it. The one exception the spec defines is an agent session (Part G §28): the coordinating node holds the session and its transcript for the session's life; participant endpoints stay stateless because the transcript travels with every turn. `;coord=` selects a coordination mode (`session`, `debate`, `roundrobin`, `freeform`); the session id is endpoint-issued (`Session-Id` header), not requestor-supplied.
- **MUST** return the envelope (Part D §17) and state the delivery mode it actually used (Part C §11.4).
- **MUST NOT** resolve `@` inside a sub-expression addressed to another node (Part B §5.6.3.1).
- **MUST** answer sync. **SHOULD** offer stream (SSE), WebSocket, and async, and advertise which (§5, §6). When asked for more than it has, it answers with the richest mode it does have; that is never an error.
- **MAY** be mounted on any node, or be its own origin. A Lambda-style function with its own URL is a node with one endpoint.
- Speaks GET. A WebSocket, when offered, is the same GET with `Upgrade: websocket` (§6). Other verbs are 405, except the ones this document adds: `DELETE` on a run handle (§6) and, if adopted, `OPTIONS` (§5).

Endpoints share nothing. An endpoint knows another endpoint only by address, and only because the expression named it.

4. Node contract

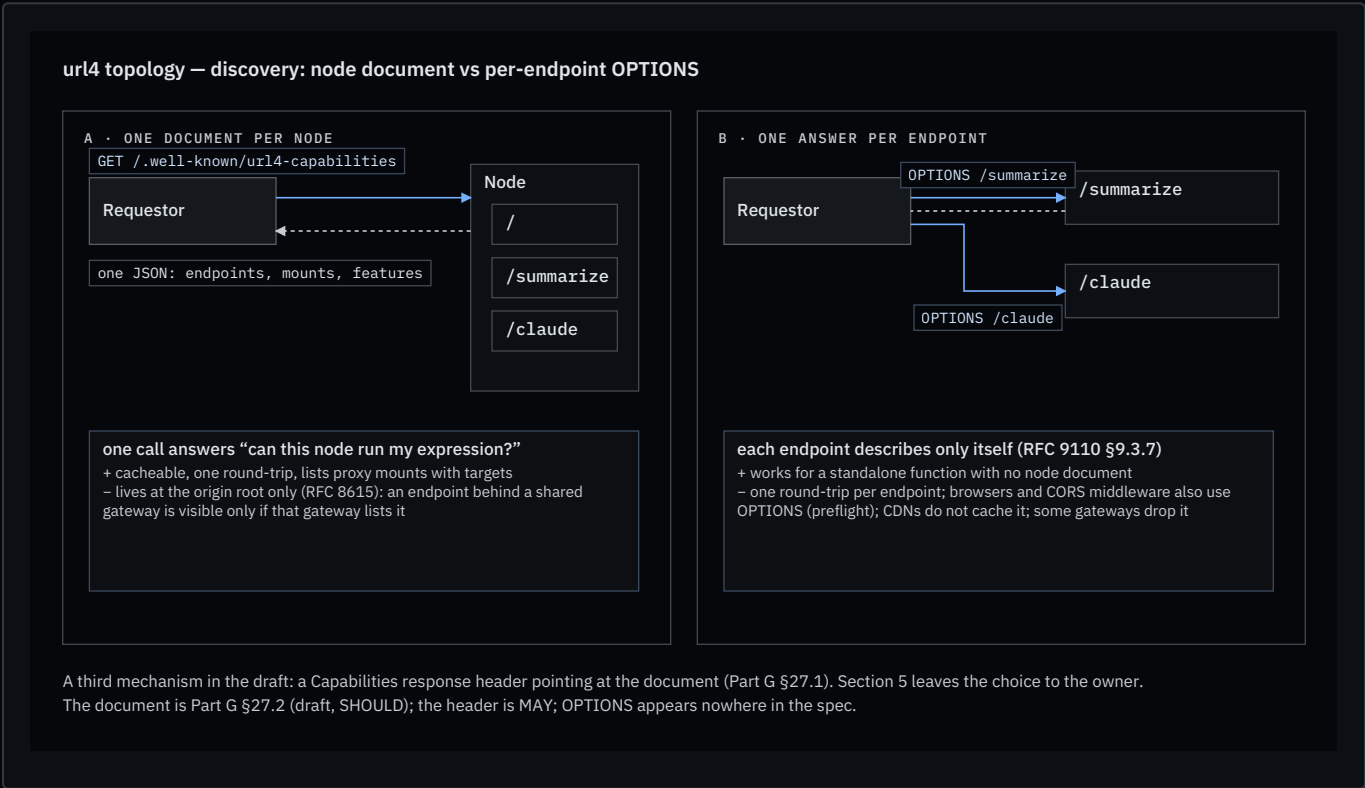
A node is an origin. It routes; it does not evaluate.

- Mounts one or more endpoints at paths. `/` is the default endpoint.
- Mount kinds: `local` (a function in the process), `command` (a subprocess), `proxy` (forwards to a declared target on another node). **Part A §1.4.3 adopted Mount and these three kinds verbatim**, alongside the spec's own intent-processor types `internal / function`, `code` and `abc_delegate` (Part G §27.3). One correction to pass back: the adopted row annotates `command` as "(subprocess, N4)", but N4 is a doctrine item in `.claude/skills/url4-engine/SKILL.md`, not a spec anchor (Appendix A delta 15). A proxied relative call is exactly the spec's delegation: the caller has already resolved the context, so what crosses the wire is materialised sources plus the intent, never a raw sub-expression. A `command` mount is code execution and inherits the pinning and signing rules of Part H §36.
- Declares proxy mounts with their targets in its capabilities document. Attribution and consumer disclosure (Part H §29.2.1) need the real target, so proxies are never hidden; a source may also cap redistribution depth or name permitted consumers (Part H §29.2.2), and a proxy target is a consumer.
- Serves discovery (§5) and the version alias `/v1` (§2).
- A local SDK process that mounts remote endpoints under local names is a node. That is how "run the ensemble on my laptop, but call `/claude` as if it were mine" works.

5. Discovery — the open decision

Three ways for a requestor to learn what a node can do. The draft Part G specifies the first two: a capabilities document (SHOULD) and a `Capabilities` response header that MAY point to it (Part G §27.1). `OPTIONS` is ours; it appears nowhere in the spec. The spec also reaches for `.well-known` for the policy registry (`/ .well-known/abc-policy`, Part H §30.2), so two well-known documents will coexist.

One thing settled since 2026-09-08. Part A §1.4.5 now writes the path as `/.well-known/url4-capabilities` — the url4-named form this document has used throughout, where Part G still says `abc-capabilities`. That is Kevin's open question 19 (Part I §43) resolving in passing for this one document. §1.4.5 also states the `Capabilities` header in the same row, which is mechanism C below; the choice between A, B and C is still the owner's, and the recommendation stays A + C.



	A · NODE DOCUMENT GET /.WELL-KNOWN/URL4-CAPABILITIES	B · PER ENDPOINT OPTIONS /<ENDPOINT>	C · CAPABILITIES RESPONSE HEADER
"Can this node run my expression?"	one call, whole answer	one call per endpoint named in the expression	pointer only; still one fetch of A
Standalone function with its own origin	works: the document sits at that origin's root	works	works
Endpoint behind a shared, non-url4 gateway	invisible unless the gateway lists it (RFC 8615: root of the origin only)	works: the endpoint answers for itself	works: any ordinary response can carry the pointer
Caching	ordinary GET: ETag , CDN, browser cache	not cached by CDNs or browsers	rides on the cached response
Browsers and CORS	plain GET	preflight uses the same verb; CORS middleware often answers first (RFC 9110 §9.3.7 allows a body, browsers ignore it)	plain header
Gateways and serverless platforms	fine	some strip or auto-answer OPTIONS	fine
Proxy mount targets (attribution)	listed in one place	per endpoint	via A

	A · NODE DOCUMENT GET /.WELL-KNOWN/URL4-CAPABILITIES	B · PER ENDPOINT OPTIONS /<ENDPOINT>	C · CAPABILITIES RESPONSE HEADER
Spec status	Part G §27.1, SHOULD; schema Part G §27.2	not in the spec	Part G §27.1, MAY

Both share one hazard: the Engine's JWT header is called `URL4-Capability`. The document is "capabilities". Appendix B proposes renaming the header.

Capabilities document: the spec's shape, plus our additions. Part G §27.2 already defines the top level: `abc_version`, `node` (one canonical URI: our node), `self_ref_support`, `collections[]` (`id`, `path`, `description`, `element_schema`, `formats`, ...), `behaviors`, `intent_processors[]` (`id`, `type` ∈ `internal` | `abc_delegate` | `http_delegate` | `code` | `function`, `endpoint`, `capabilities`), `processor_policy`, `mode_support`, `agent_profiles`. Content types come from Part F §25.4 as `consumes` / `produces`. Sub-paths are read as collections (Part G §27.4). Our additions, marked, are what makes a processor addressable as an endpoint:

```
{
  "abc_version": 1,
  "node": "url4://alpha.example",
  "default_processor": "summarize", // ours
  "intent_processors": [
    { "id": "summarize", "type": "internal", "path": "/summarize", // path: ours
      "consumes": ["text/plain"], "produces": ["text/plain"],
      "delivery": ["sync", "stream"] }, // delivery: ours
    { "id": "upper", "type": "code", "path": "/upper" },
    { "id": "claude", "type": "abc_delegate", "path": "/claude",
      "endpoint": "url4://beta.example/claude" }
  ],
  "schemes": ["s3", "pg", "sqlite"], // ours (§2)
  "collections": [ { "id": "science", "path": "/science", "self_ref_eligible": true } ],
  "self_ref_support": true
}
```

One point to settle with Kevin: the spec's document describes one node whose processors are addressed by `id` and whose sub-paths are collections; our endpoints need a path each. `intent_processors[].path` is the smallest change that gives both.

Owner decision (open). Pick one:

☐

A only — node document is a MUST; nothing per endpoint.

☐

B only — OPTIONS per endpoint is a MUST; no node document.

☐

A MUST + C SHOULD — the spec's pair, one level stronger than its SHOULD/MAY.

☐

A MUST + B SHOULD — the document indexes the node; an endpoint may also answer OPTIONS with its own entry.

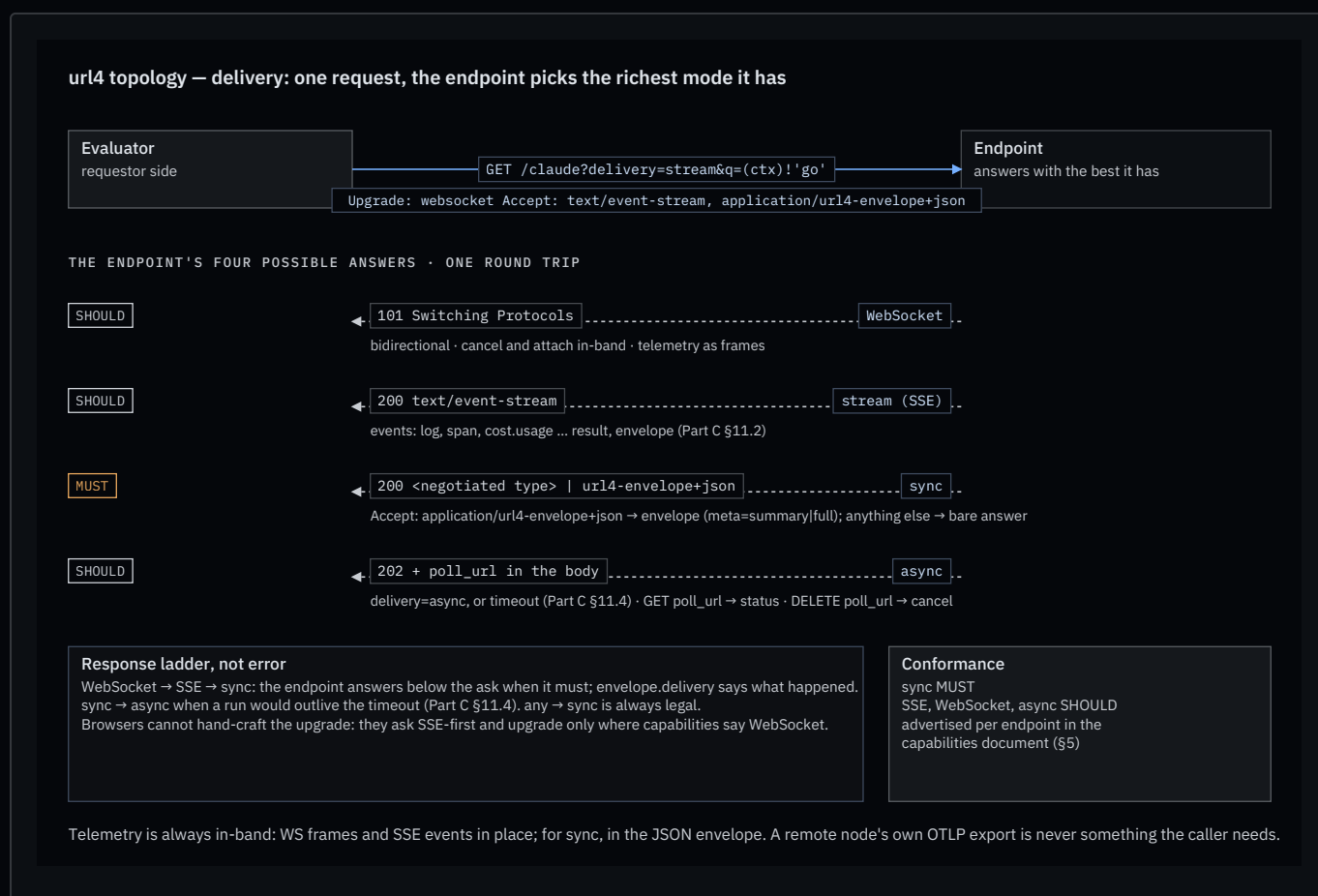
Recommendation, revised after reading Part G: **A MUST + C SHOULD**, B dropped. The `Capabilities` header solves the gateway case that motivated OPTIONS, and it is already in the draft.

6. Delivery and transport

This section is no longer a proposal. Part A §1.4.4 has adopted both halves of it. **Delivery mode** now lists four values — **sync** (one body), **stream** (SSE on the same GET), **async** (202 plus **poll_url**) and **websocket** (101 on the same GET) — with **sync** the only one every endpoint MUST support. And the **response ladder** is now a spec term in its own right: "the mechanism by which a node responds with the richest delivery mode it supports, WebSocket → SSE → sync... answering below a requested mode level is reported as a degradation, not an error", recorded as an extension of Part C §10.2. **Degradation** itself is likewise now defined (§1.4.4).

Two things to carry forward. Kevin's own row is annotated "**SPEC SECTION NEEDS UPDATING**" — Part C §11 still describes three modes and two ladder edges, so §1.4.4 and Part C disagree until that lands; Appendix A delta 4 is now about closing that gap rather than proposing the ladder. And both new rows say a "**host**" responds, where it is the node that negotiates and answers (Appendix A delta 13).

Part I §43 question 26 rules **ws://** sources out of scope; that is a different question from a WebSocket answer on the endpoint's own GET, and adopting **websocket** as a delivery value settles it in our favour.



One request, the endpoint picks. The evaluator sends the richest ask it can, once:

```
GET /claude?delivery=stream&q=(ctx)!'go'
Upgrade: websocket
Accept: text/event-stream, application/json
```

The endpoint answers with the best it supports. RFC 6455 lets a WebSocket handshake ride an ordinary GET; RFC 9110 lets a server ignore **Upgrade** and answer normally. So one round trip covers the whole response ladder:

ENDPOINT ANSWERS	MEANING	TELEMETRY TRAVELS AS
101 Switching Protocols	WebSocket : bidirectional; cancel and attach in-band	frames, same names as the SSE events
200 text/event-stream	stream : SSE on the same GET (Part C §11.2)	SSE events, then result , then envelope
200 text/plain or application/json	sync : bare answer, or the envelope, by Accept	envelope meta (below)
202 + poll_url	async : delivery=async ; the 202 body carries job_id , rid , poll_url (Part C §12.5); GET it for status, DELETE it to cancel	on the handle

Rules:

- **sync is the only MUST.** SSE, WebSocket and async are SHOULD, advertised per endpoint in the capabilities document (§5). A serverless function that offers only sync and SSE is conformant.
- **Response ladder, not error.** WebSocket → SSE → sync. An endpoint answering below the ask is normal; the envelope's **delivery** field says what happened (Part C §11.4), and the degradation is reported like any other (Part C §10.2.2). Part C §11.4 still lists only two edges, **stream → sync** and **sync → async**; Part A §1.4.4 now carries the full WebSocket → SSE → sync ladder and the **any → sync** rule, so the two parts disagree until Part C is updated.
- **Async is one of the spec's four delivery values** (§1.4.4: **sync**, **stream**, **async**, **websocket**). It is requested with **delivery=async** (Part C §9.1), or entered by the endpoint when a sync run would outlive the timeout (Part C §11.4 **sync → async**). The Engine's **Prefer: respond-async** becomes an alias. The run handle is the **poll_url** field of the 202 body; a **Location** header mirroring it is our addition.
- **Browsers go SSE-first.** A browser cannot attach **Accept** to a WebSocket handshake, so a browser evaluator asks for SSE and upgrades only where the capabilities document says WebSocket is offered.
- **The Engine's product session** keeps its WebSocket at **/ws**. It is one instance of the WebSocket rung, not a separate protocol.
- **Telemetry is always in-band.** Whatever the mode, the caller receives logs, spans and cost on the same connection or in the envelope. A remote node's own OTLP export is its business, never something the caller depends on (§7).

Telemetry per mode. WebSocket and SSE carry it in place: an SSE body is a sequence of named events (**event: log**, **event: span**, **event: cost.usage**, then **event: result**, **event: envelope**), in order, on the one response. The spec already builds on this (Part C §12.5 is an SSE event catalog); our three signals are three more names. What SSE lacks against WebSocket is only the return path: no in-band cancel or attach.

Sync has no room for that: the body is the answer. The spec always returns the JSON envelope and uses **Accept** to negotiate the result's content type (Part C §9.1, Part F §25.2). We add one wrapper switch on top, named so it cannot collide with that:

KNOB	GOVERNS	VALUES
Accept header	the wrapper, by one dedicated type	application/url4-envelope+json : the envelope (Part D §17). Anything else: the bare answer in the negotiated content type, no logs, no telemetry, what url4 serve returns today. Adopted — Part A §1.4.4's Envelope row now states exactly this
meta param	how much the envelope carries	none : result and status. summary : counts, latency, cost. full : per-source detail, nested child envelopes (Part D §17.3), and a telemetry block with logs and spans. The telemetry block is ours; per the Hybrid Rule it is present and null at summary (Part D §17.2.2)
fmt param	the shape of the answer content	text , markdown , json (Part C §9.1); orthogonal, a JSON envelope can wrap a markdown answer

An envelope answer with no `meta` gets `summary` (the spec's default is `none`, Part C §9.1), so asking for the envelope always buys the cheap insight; `meta=full` is the explicit debug switch. What an endpoint exposes is its decision (Part D §18.4): a production endpoint may answer `meta=full` with the `telemetry` block redacted, but it MUST keep the structure and use `null` or `"redacted"`, never drop fields silently. A development endpoint hands over everything. Same envelope, same evaluator code, different policy.

Cancel. The spec calls cancellation a gap and offers two shapes, `DELETE <poll_url>` or a `cancel=<rid>` parameter (Part C §16.2). We take the first: over WebSocket, cancel is in-band; otherwise `DELETE` on the `poll_url`, which the Engine already does. New terminal state `cancelled`; SSE event `request.cancelled`; partial result returned when quorum was already met, as §16.2 asks. Note for Kevin: `status: "cancelled"` is not yet in the status enum of Part D §17.4.

7. Telemetry

Three signals, one trace, as in the doctrine skill. What changes is where a one-shot GET puts them.

Part A §1.4.3 now defines **attribution** in the terms this document used — "a per-source influence score of the impact of each source on the response body, reported in the envelope, shaped by the request's source weights and budgets" — so what follows sits on a defined word rather than an implied one.

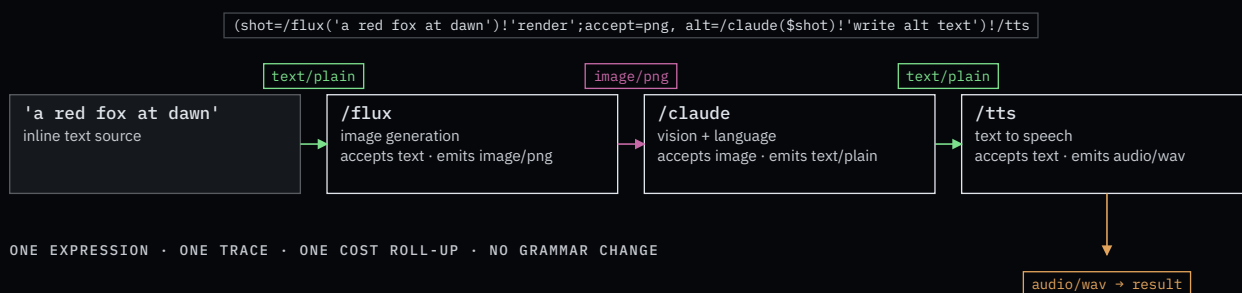
- **sync:** in the body, only when the caller asked for the envelope (`Accept: application/url4-envelope+json`, §6). `meta` sets the level (`summary` by default, `full` adds logs and spans; Part D §17.2). At `meta=full` a child's envelope nests in `source.envelope` (Part D §17.3). Each endpoint aggregates only what it saw itself (Part D §18.1). A bare `text/plain` answer carries nothing.
- **stream** and **WebSocket:** as events: logs, spans, `cost.usage` (self and subtree), then `result`, then `envelope`. Same names in both.
- **durable:** OTLP export to the trace backend. There is no separate "Enclave" store; the exporter superseded it.
- **What the spec already carries:** 21 SSE event types (Part C §12.5), `meta.total_cost` and `budgets_spent` in the envelope (Part D §17.1.2.8, Part E §24), and an endpoint-local audit log (Part H §34). Our logs, spans and `cost.usage` events are additions on top of that catalog, not replacements; cost stays a separate event, and `total_cost` is its roll-up. Every streamed event with content already MUST carry `content_type` (Part C §12.5), which is what §8 builds on.

This closes doctrine item F4. **Idempotency:** a url4 GET is idempotent in the HTTP sense (RFC 9110 §9.2.2): repeating it has no extra server-side effect. Results need not be identical; Part C §15.1 conflates the two (Appendix A). One real exception the spec lists: a repeat that would exceed a consumed budget fails with `budget_exceeded` (Part C §15.2).

8. Typed payloads: an endpoint is a universal inference processor

We have treated an endpoint as text in, text out. That is a habit, not a spec rule. The spec defines a source as "a URI, text, or nested expression" and an intent processor as whatever executes the intent on behalf of the endpoint (Part A §1.4). Nothing says the payload is a string.

url4 topology — typed payloads: every edge carries a media type



How a binary result travels

small: inline — result.content as a data URI + result.content_type (Part G §26.2.3), or the bare response body with its Content-Type
 large: by reference — result.artifact, an https:// data URL
 the next endpoint fetches like any source (Part B §3.5)
 threshold: the endpoint's, advertised as inline_max_bytes

How an endpoint declares types

source: fetched Content-Type · text/plain · emitting endpoint's type
 ;accept=<alias> says what the consumer wants (Part B §4.2, Part F §25.5)
 ct_mismatch says what to do when it differs (Part B §8.1.2)
 capabilities: consumes: [...], produces: [...] per processor (Part F §25.4)
 an evaluator can type-check the expression before spending

Text stays the default. Image generation, speech, video and multimodal ensembles become ordinary url4 expressions on the same engine.
 Over WebSocket a binary frame carries bytes; over SSE a large payload always goes by reference.

Proposal. Every edge in an expression carries a **typed payload**, the way a ComfyUI edge carries an image, a latent or a mask. The type system is the media type: `text/plain`, `image/png`, `audio/wav`, `video/mp4`, `application/json` for embeddings and structured data. Text stays the default and the most common type. No grammar change.

```
(shot=/flux('a red fox at dawn')!'render';accept=png,
alt=/claude($shot)!'write alt text')!/tts
```

Edges: text → image → text → audio. Three endpoints, three processors, one expression, one trace, one cost roll-up.

What the spec already gives us

- `;accept` is a source-level execution annotation (Part B §4.2) that takes a **short alias** (`json`, `csv`, `markdown`, ...), never a MIME type, because `/` would read as a relative URI (Part F §25.5). The registry has no image, audio or video rows yet; `png`, `jpeg`, `wav`, `mp4` are proposed in Appendix A. `ct_mismatch` has four values, `fail` | `ignore` | `transform_ignore` | `transform_fail`, default `ignore`, and only applies when `;accept` was declared (Part G §26.3). The spec calls input negotiation out of scope and treats an unusable format as an intent-execution failure, not a source failure (Part F §25.10).
- The spec already has a weighted `image/png` source feeding a text intent (Part I §41.20), and Part G §26.2.3 already says how binary content travels in LLM mode: a data URI, `data:<mediatype>;base64,...`, with raw bytes for RDS intents. The SDK carries a media type on every fetch (`FetchRequest.media_type`) and parses collections by declared type (Part B §5.3.7). The Engine already forks results by size: inline, spilled to a content-addressed artifact, or refused above a hard cap.

Three rules the spec does not yet give (Part F, §25)

1. **A source declares its type.** A URI source has the `Content-Type` it was fetched with; absent that, the endpoint infers from the path or sniffs, and falls back to `application/octet-stream` (Part G §26.2). Inline text is `text/plain`. A nested expression has the `result.content_type` its endpoint emitted (Part D §17.1.2.1). `;accept` says what the consumer wants; `ct_mismatch` says what to do when the two differ.

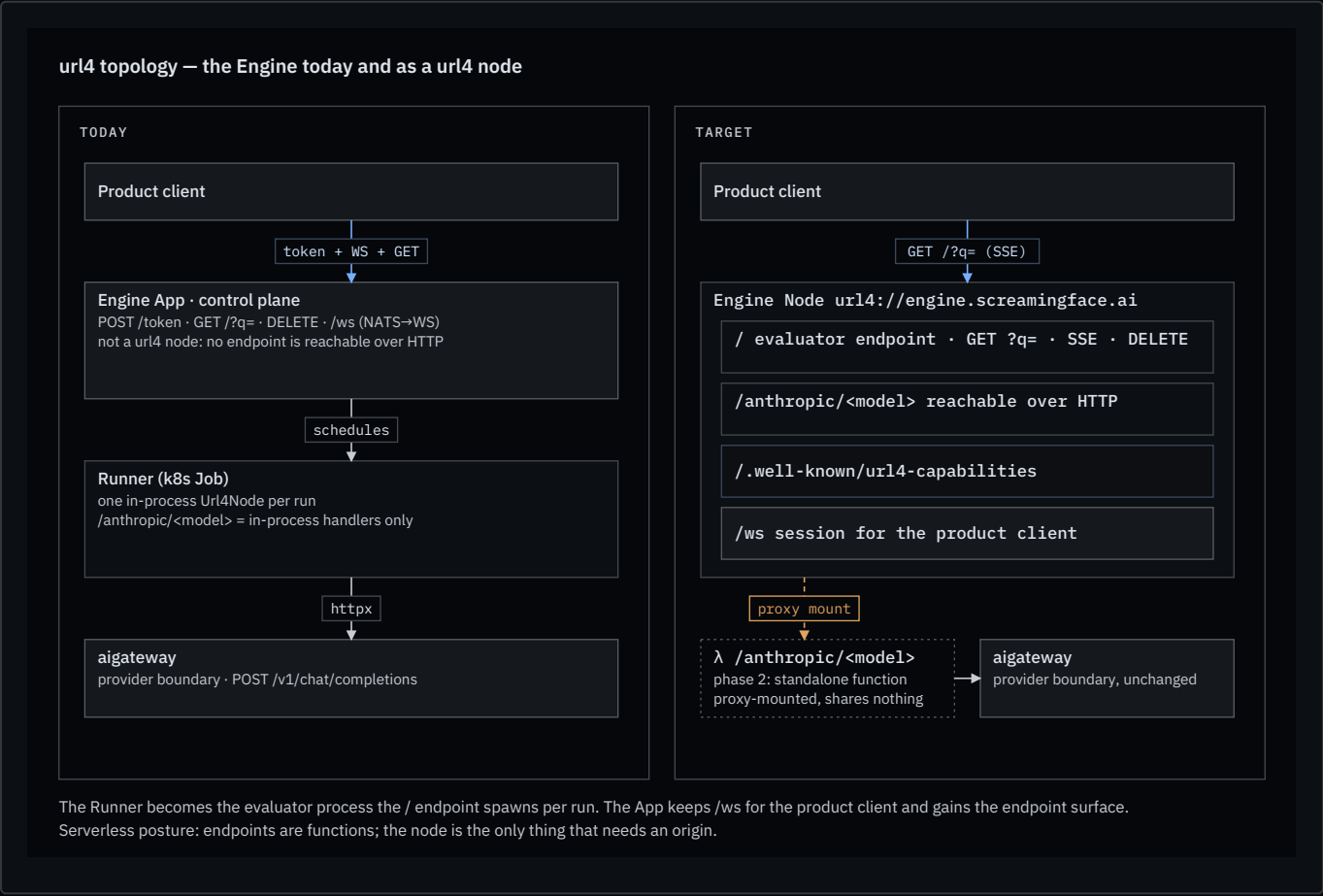
2. **A binary result travels inline when small, by reference when large.** Bare sync: the response body is the bytes, with its `Content-Type` (Part F §25.5 permits raw binary outside the envelope). Envelope: `result.content` carries a small payload as a data URI with `result.content_type` set, the spec's own form; a large payload becomes `result.artifact`, an `https://` URL that the next endpoint fetches as plain data (Part B §3.5). The inline threshold is the endpoint's, advertised in its capabilities. Over WebSocket a binary frame carries the bytes; over SSE a large payload always goes by reference. Two cautions from the drafts: this is not truncation, which Part G §26.2.1 forbids; and an artifact URL is a redistribution, so it inherits the source's flow constraints and depth cap (Part H §29.2.2).
3. **A processor advertises what it accepts and emits.** The spec's names are `consumes` and `produces` (Part F §25.4), per processor in the capabilities document. An evaluator can then type-check an expression before it spends anything, which is what the dry-run question in §10 builds on.

```
{ "id": "flux",    "type": "internal", "path": "/flux",
  "consumes": ["text/plain"], "produces": ["image/png"], "inline_max_bytes": 262144 },
{ "id": "claude", "type": "internal", "path": "/claude",
  "consumes": ["text/plain", "image/png", "image/jpeg"], "produces": ["text/plain"] },
{ "id": "tts",    "type": "internal", "path": "/tts",
  "consumes": ["text/plain"], "produces": ["audio/wav"] }
```

Why it matters. Image generation, text-to-speech, speech-to-text, video evaluation and multimodal ensembles all become ordinary url4 expressions on the same engine, with the same telemetry, the same cost accounting and the same attribution. aigateway stays the provider boundary; it already fronts image and audio providers. The Engine's artifact store is rule 2 as built.

Open. A media type for embeddings (`application/json` with a profile, or a vendor type); how attribution weights and `tokens=` budgets apply to non-text sources (Part E is silent; no `bytes=` or `seconds=` budget key exists). Settled by the draft: `fmt` stays a structure hint and a future version MAY retire it in favour of `Accept` (Part F §25.8).

9. The Engine as a node



Appendix C — where the spec lives

- Parts A and B, v0.5 DRAFT (2026-07-10): `secondbrain/kevin-mcdonough/docs/adrs/URL4-Spec-A.md`, `URL4-Spec-B.md`.
- **Part A §1.4 as cited throughout this document** is not that file but the version in `OpenMined/screamingface-design PR #19` (`url4/terminology-refresh`, commit `2a939bff`, opened 2026-09-15), which is a **draft against** `main` and has had no review. Every "`§1.4.x`" anchor here resolves there and nowhere else yet.
- Parts C–I: draft markdown on the `url4-refactor` **branch** of `OpenMined/screamingface-design`, `kevin-mcdonough/docs/adrs/refactor/URL4-Spec-{C...I}.md`, cut 2026-04-28 from the v0.4 text, not yet reviewed, never merged to `main` (where the site shows "Not yet written" stubs). Cited here as "Part X §N". The same branch holds the v0.4 monolith (commit `f28608a`); the v0.2 monolith this document was first written from is `8a052dc`, and its numbering diverges from §21 on.
- Public docs: `public-docs/src/pages/learn/Url4Page.vue`. They use "fusion" and "typed DAG"; the spec uses neither.
- Doctrine: `.claude/skills/url4-engine/SKILL.md`, updated with this document.